

Formal Semantics for Knowledge Representation and Automated Reasoning in BPMN Process Models

Angelo Casciani^a, Simone Agostinelli^b,
Yves Lespérance^c, Andrea Marrella^a, Sebastian Sardiña^d



SAPIENZA^a
UNIVERSITÀ DI ROMA



YORK^c



RMIT^d
UNIVERSITY



Finanziato
dall'Unione europea
NextGenerationEU



Ministero
dell'Università
e della Ricerca



Italiadomani
PIANO NAZIONALE
DI RIPRESA E RESILIENZA



BPM 2026 – Journal First Track

Toronto, Canada

September 29, 2026

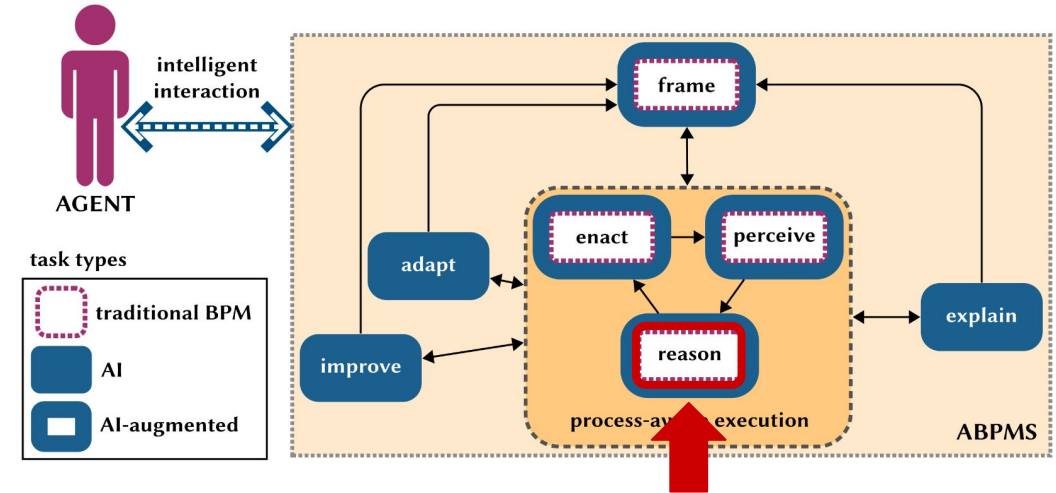


Published in:
Information Systems,
Volume 140, 2026, 102718

BPMN's Missing Formal Foundation

BPMN is the standard for process modeling

- However, its execution semantics is described in **natural language**
- This leads to **variability** across BPM systems: data flow, error handling, collaboration are open to interpretation
- Increased risk of **semantic errors**



Why does it matter now?

- **AI-augmented BPMs** demands a **reasoning layer** that interprets model's semantics and decides the next action.
- That layer needs a **formal specification** of BPMN, one that handles data and exceptions, not just control-flow.

- Business Process Modeling Notation (BPMN) v2.0: <http://www.omg.org/spec/BPMN/2.0/>
- M. Weske. Business Process Management. Concepts, Languages, Architectures, Springer, 2024.
- R.M. Dijkman et al. Semantics and Analysis of Business Process Models in BPMN, Inf. Softw. Technol. 50 (12) (2008).
- M. Dumas et al., AI-augmented Business Process Management Systems: A Research Manifesto, ACM Trans. Manag. Inf. Syst. 14 (1) (2023) 1–19.

Two Research Questions

RQ1

How can Knowledge Representation and Reasoning languages formalize the semantics of BPMN?

RQ2

How can this encoding enable data-aware reasoning that goes beyond structural analysis?

Situation Calculus and ConGolog

Situation Calculus

- A dynamic world evolving through **situations**, recording every action performed so far, from the initial situation S_0 .
- **(Relational) Fluents** capture what changes: relations whose truth value depends on the current situation.
- A **basic action theories** define when an action is legal (*action precondition axioms*) and what it changes (*successor state axioms*).

ConGolog

- A high-level program language over situation calculus, with **sequence, choice, iteration**, and, importantly for BPM, **concurrency** and **interrupts**.
- Concurrent, multi-threaded execution is what business process models with pools and parallel gateways need.

- R. Reiter. Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems, MIT Press, 2001.
- G. De Giacomo et al. ConGolog, a concurrent programming language based on the situation calculus, Artificial Intelligence 121 (1) (2000) 109–169.

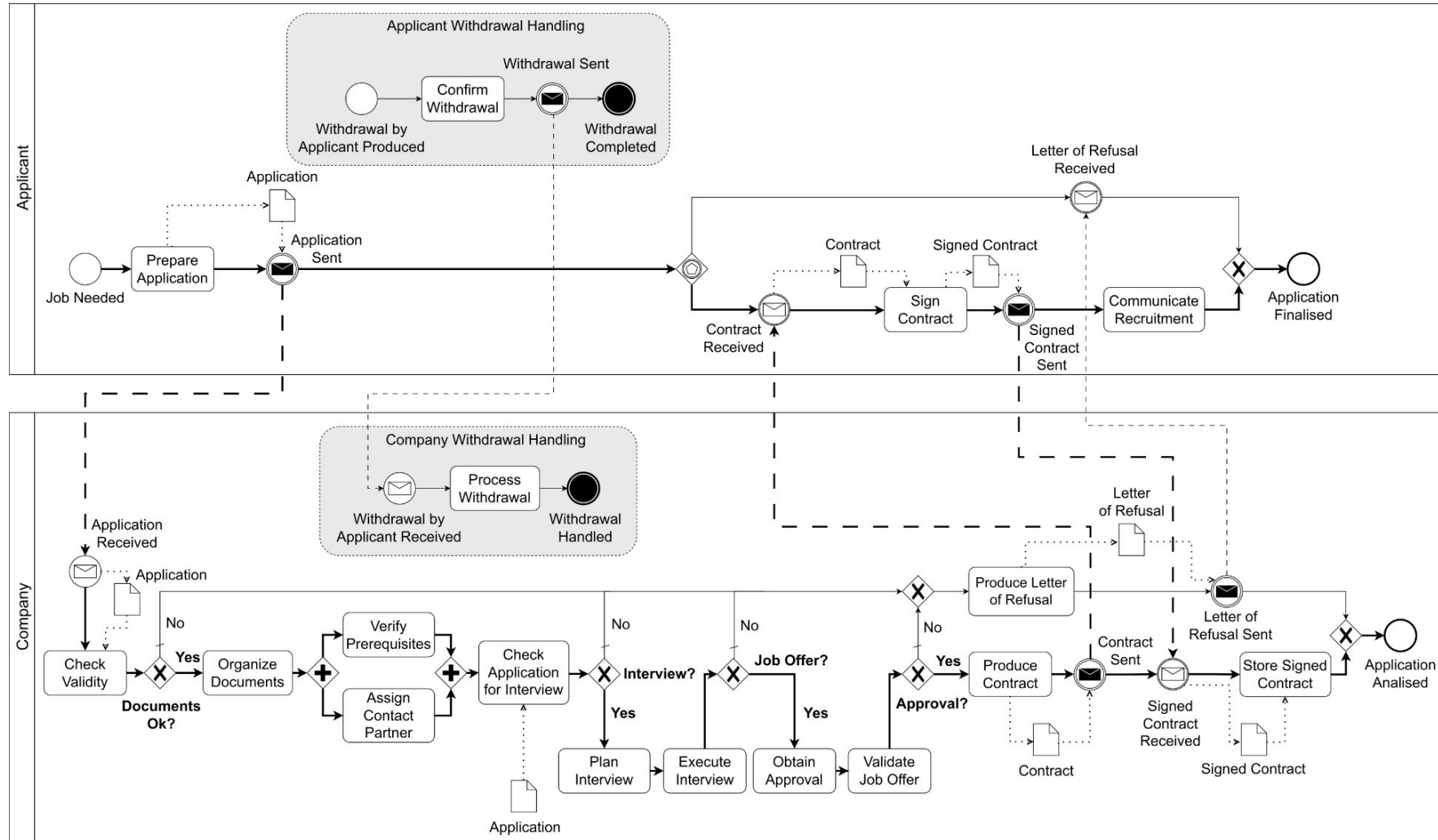
Formalizing BPMN Patterns in ConGolog

PATTERN	HIGH-LEVEL PROGRAM	BASIC ACTION THEORY
	$(\alpha_s(id, \vec{x}); \text{Done}(\alpha_s(id, \vec{x})))?$	$\alpha_s, \alpha_e \in \mathcal{A}. \text{Exog}(\alpha_e). \forall x \in \vec{x}. \beta(id, x, s), \forall y \in \vec{y}. \gamma(id, y, s),$ $\text{Running}(\alpha_s(id, \vec{x}), s), \text{Done}(\alpha_s(id, \vec{x}), s) \in \mathcal{F}.$ $\text{Poss}(\alpha_s(id, \vec{x}), s) \equiv \neg \text{Running}(\alpha_s(id, \vec{x}), s) \wedge \beta(id, \vec{x}, s).$ $\text{Poss}(\alpha_e(id, \vec{y}), s) \equiv \text{Running}(\alpha_s(id, \vec{x}), s) \wedge \gamma(id, \vec{y}, s).$ $a = \alpha_s(id, \vec{x}) \supset \text{Running}(\alpha_s(id, \vec{x}), \text{do}(a, s)).$ $a = \alpha_e(id, \vec{y}) \supset \text{Done}(\alpha_s(id, \vec{x}), \text{do}(a, s)).$ $\text{Running}(\alpha_s(id, \vec{x}), S_0) = \text{false}.$ $\text{Done}(\alpha_s(id, \vec{x}), S_0) = \text{false}.$
	$(\delta_i; \delta_{i+1})$	-
	if ϕ then δ_i else δ_j endIf	-
	$\delta_j \parallel \delta_k$	-
	$(\delta; \text{while } \phi \text{ do } \delta \text{ endWhile})$	-
	$\epsilon_t(id, \vec{x})$	$\epsilon_t \in \mathcal{A}. \text{Done}(\epsilon_t(id, \vec{x}), s), \forall x \in \vec{x}. \beta(id, x, s) \in \mathcal{F}.$ $\text{Poss}(\alpha_s(id, \vec{x}), s) \equiv \beta(id, \vec{x}, s).$ $a = \epsilon_t(id, \vec{x}) \supset \text{Done}(\epsilon_t(id, \vec{x}), \text{do}(a, s)).$ $\text{Done}(\epsilon_t(id, \vec{x}), S_0) = \text{false}.$
	$(\text{Done}(\epsilon_i(id)); \delta_i) \mid (\text{Done}(\epsilon_j(id)); \delta_j)$	$\text{Done}(\epsilon_i(id), s), \text{Done}(\epsilon_j(id), s) \in \mathcal{F}.$

PATTERN	HIGH-LEVEL PROGRAM	BASIC ACTION THEORY
	-	$\epsilon_s \in \mathcal{A}. \text{Exog}(\epsilon_s). \text{Active}(id, s), \forall y \in \vec{y}. \beta(id, \vec{y}, s),$ $\text{Done}(\epsilon_s(id, \vec{x}), s), \text{Pool}(\rho, s), \text{Waiting}(id, \rho, s) \in \mathcal{F}.$ $\text{Poss}(\epsilon_s(id, \vec{x}), s) \equiv \neg \text{Active}(id, s) \wedge \beta(id, \vec{y}, s).$ $a = \epsilon_s(id, \vec{x}) \supset \text{Active}(id, \text{do}(a, s)).$ $a = \epsilon_s(id, \vec{x}) \supset \text{Done}(\epsilon_s(id, \vec{x}), \text{do}(a, s)).$ $a = \epsilon_s(id, \vec{x}) \supset \text{Waiting}(id, \rho, \text{do}(a, s)).$ $\text{Active}(id, S_0) = \text{false}. \text{Done}(\epsilon_s(id, \vec{x}), S_0) = \text{false}.$ $\text{Waiting}(id, \rho, S_0) = \text{false}.$
	$\epsilon_e(id, \vec{x})$	$\epsilon_e \in \mathcal{A}. \text{Active}(id, s), \text{Done}(\epsilon_e(id, \vec{x}), s), \forall x \in \vec{x}. \beta(id, \vec{x}, s) \in \mathcal{F}.$ $\text{Poss}(\epsilon_e(id, \vec{x}), s) \equiv \text{Active}(id, s) \wedge \beta(id, \vec{x}, s).$ $a = \epsilon_e(id, \vec{x}) \supset \neg \text{Active}(id, \text{do}(a, s)).$ $a = \epsilon_e(id, \vec{x}) \supset \text{Done}(\epsilon_e(id, \vec{x}), \text{do}(a, s)).$ $\text{Done}(\epsilon_e(id, \vec{x}), S_0) = \text{false}.$
	proc $\rho_i \delta_i^{\parallel} \text{endProc}$ proc δ_i $\pi id. (\text{acquire}(id, \rho_i); \text{handle}_i(id))$ endProc proc $\rho \rho_1 \parallel \dots \parallel \rho_N \rangle$ $\langle \neg \text{Servers_stopped} \rightarrow \text{true?} \rangle$ endProc	$\text{acquire}, \text{shut_down} \in \mathcal{A}. \text{Exog}(\text{shut_down}).$ $\text{Servers_stopped}(s), \text{Pool}(\rho, s), \text{Waiting}(id, \rho, s) \in \mathcal{F}.$ $\text{Poss}(\text{acquire}(id, \rho), s) \equiv \text{Pool}(\rho, s) \wedge \text{Waiting}(id, \rho, s).$ $a = \text{acquire}(id, \rho) \supset \neg \text{Waiting}(id, \rho, \text{do}(a, s)).$ $\text{Poss}(\text{shut_down}, s) \equiv \neg \text{Servers_stopped}(s) \wedge \neg(\exists id. \text{Active}(id, s)).$ $a = \text{shut_down} \supset \text{Servers_stopped}(s).$ $\text{Servers_stopped}(S_0) = \text{false}.$
	gexec (ϕ, δ); if $\neg \phi$ then ($\delta_e; \epsilon_t$) else () endIf	$\epsilon_s, \epsilon_t \in \mathcal{A}. \text{Exog}(\epsilon_s). \text{Active}(id, s), \text{Done}(\epsilon, s) \in \mathcal{F}.$ $\text{Poss}(\epsilon_s(id), s) \equiv \text{true}. \text{Poss}(\epsilon_t(id), s) \equiv \text{Done}(\delta_e, s).$ $a = \epsilon_s(id) \supset \neg \phi(\text{do}(a, s)).$ $a = \epsilon_s(id) \supset \text{Done}(\epsilon_s(id), \text{do}(a, s)).$ $a = \epsilon_t(id) \supset \text{Done}(\epsilon_t(id), \text{do}(a, s)).$ $a = \epsilon_t(id) \supset \neg \text{Active}(id, \text{do}(a, s)).$ $\text{Done}(\epsilon_s(id), S_0) = \text{false}. \text{Done}(\epsilon_t(id), S_0) = \text{false}.$

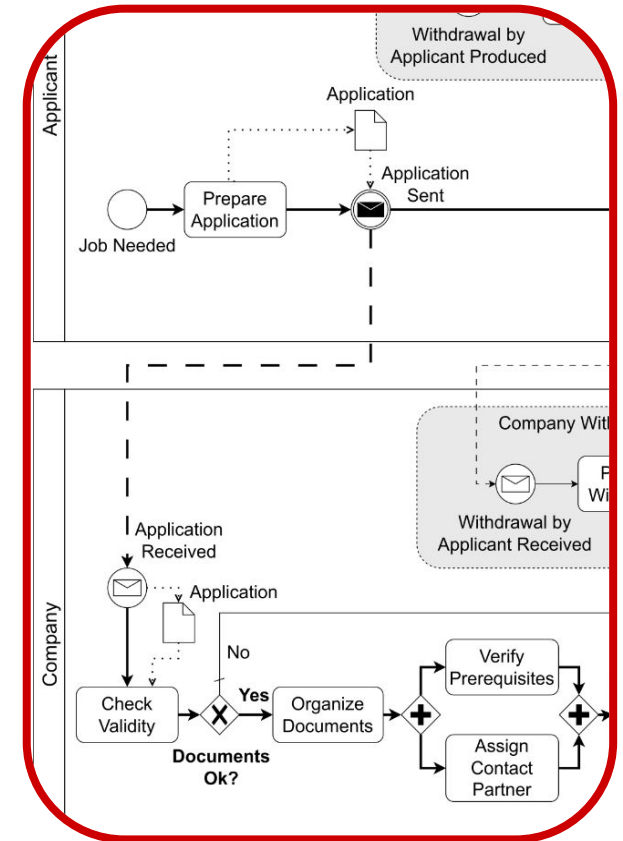
Minimal set of logical statements for efficient reasoning.

Example: A Job Application Process



Encoding the Example

- `job_needed(id) → Active(id,applicant,s)`
- `prepare_application_start(id)/prepare_application_end(id,app)`
- `application_sent(id,app) → Done(application_sent(id,app),s)`
- `check_validity_start(id,app)/check_validity_end(id,result) → Documents_ok(id,s)`
- `if Documents_ok(id,s) then [organize_documents_start(id),...]`
`else [produce_letter_of_refusal_start(id) ...]`



- G. De Giacomo et al. IndiGolog: A high-level programming language for embedded reasoning agents, in: Multi-Agent Programming, Languages, Tools and Applications, Springer, 2009.
- Case Study Implementation: https://github.com/angelo-casciani/sitcalc4bpmn/tree/main/pl_models/case_study
- IndiGolog repository: <https://github.com/ai-krml-uoft/indigolog>

A New Construct for Global Exceptions

Guarded Execution

- ConGolog can't gracefully abort a running program, yet BPMN global exceptions need that.
- We introduce **guarded execution** :
 - **gexec(φ, δ)** runs δ only while φ holds and terminates immediately once it doesn't.
- Petri nets need explicit arcs from every state to the handler.
 - **gexec(φ, δ)** captures it natively, without altering the main control flow.

Transition Semantics for **gexec(φ, δ)**

- $\text{Trans}(\text{gexec}(\varphi, \delta), s, \delta', s') \equiv \varphi[s] \wedge \exists \delta''. \text{Trans}(\delta, s, \delta'', s') \wedge \delta' = \text{gexec}(\varphi, \delta'')$
- $\text{Final}(\text{gexec}(\varphi, \delta), s) \equiv \neg \varphi[s] \vee \text{Final}(\delta, s)$

Guarded Execution in Action

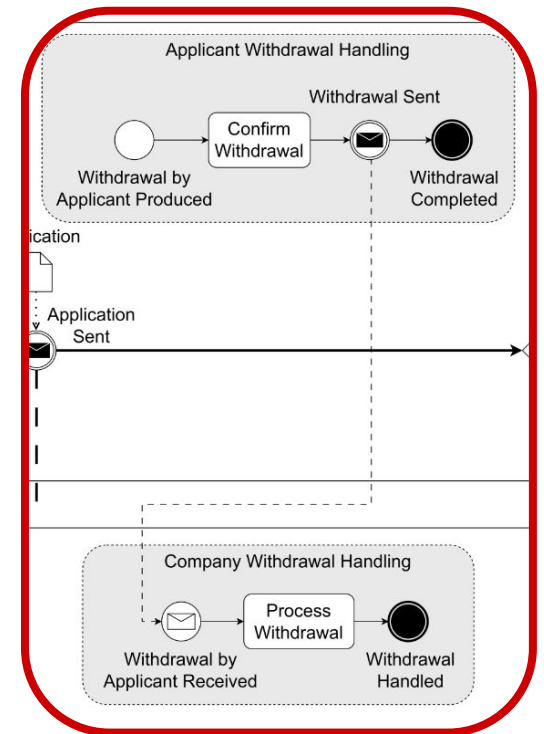
```
[gexec(Active(id,applicant)  $\wedge$   $\neg$ Done(withdrawal_by_applicant(id),
applicant_procedure(id)),
if Active(id,applicant)  $\wedge$  Done(withdrawal_by_applicant(id)
then withdrawal_app_procedure(id)
else [] ]
```

APPLICANT POOL

||

```
[gexec(Active(id,company)  $\wedge$   $\neg$ Done(withdrawal_sent),
company_procedure(id))
if Active(id,company)  $\wedge$  Done(withdrawal_sent)
then withdrawal_company_procedure(id)
else [] ]
```

COMPANY POOL



Supported Reasoning Tasks



Projection

Does a fluent hold after a given action sequence?



Legality

Is a given execution sequence executable?



Conformance Checking

Can an execution trace be reproduced by the model?



Property Verification

Does some legal execution satisfy/violate a property of interest?

- R. Reiter. Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems, MIT Press, 2001.
- G. De Giacomo et al. ConGolog, a concurrent programming language based on the situation calculus, Artificial Intelligence 121 (1) (2000) 109–169.

Finding an Unsafe Race Condition

Can a contract end up signed while the application was withdrawn?

- Property verification finds a history: so, **yes**.
- The company can commit to a contract even after the applicant has withdrawn.

```
?- time(do(property_verification, [], H)).
% 22,427,074 inferences, 1.117 CPU in 1.117 seconds (100% CPU, 20076892 Lips)
H = [end_bpmn, withdrawal_handled(1), process_withdrawal(end, 1), process_withdrawal(start, 1), withdrawal_completed(1), withdrawal_sent(1), confirm_withdrawal(end, 1), confirm_withdrawal(start, 1), withdrawal_by_applicant(...)|...] .
```



Detected in ~ 1 second



Requires guards +
concurrency + interrupts

From BPMN to IndiGolog: Tool

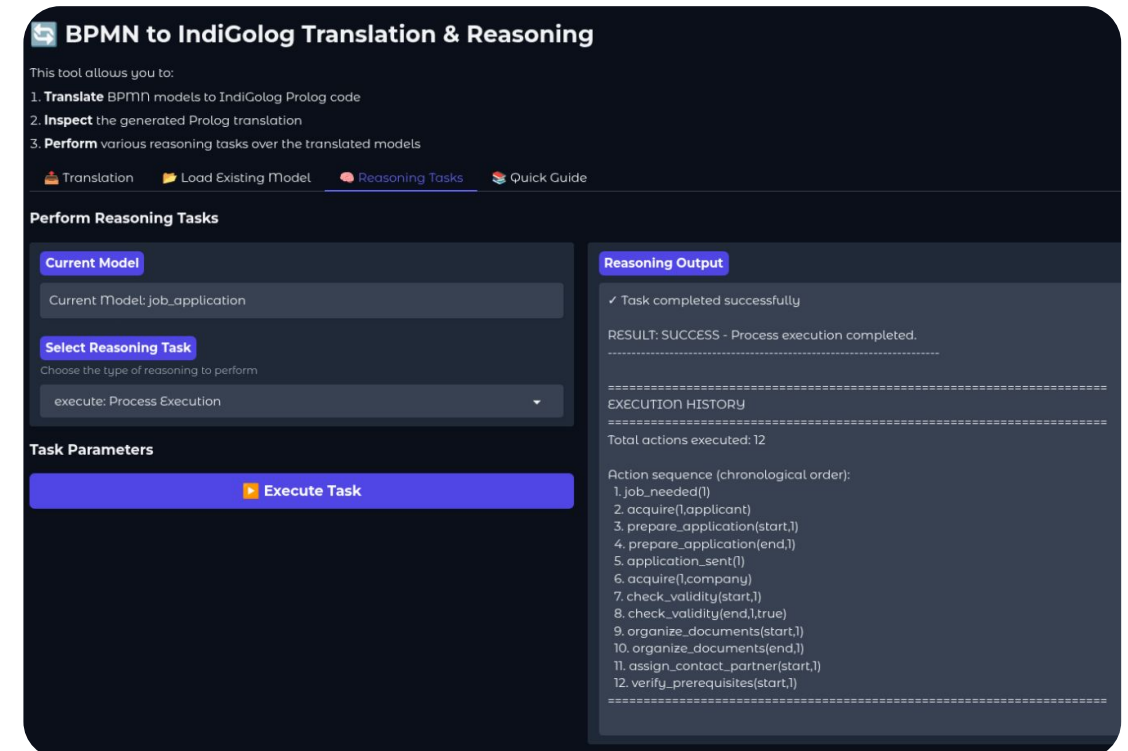
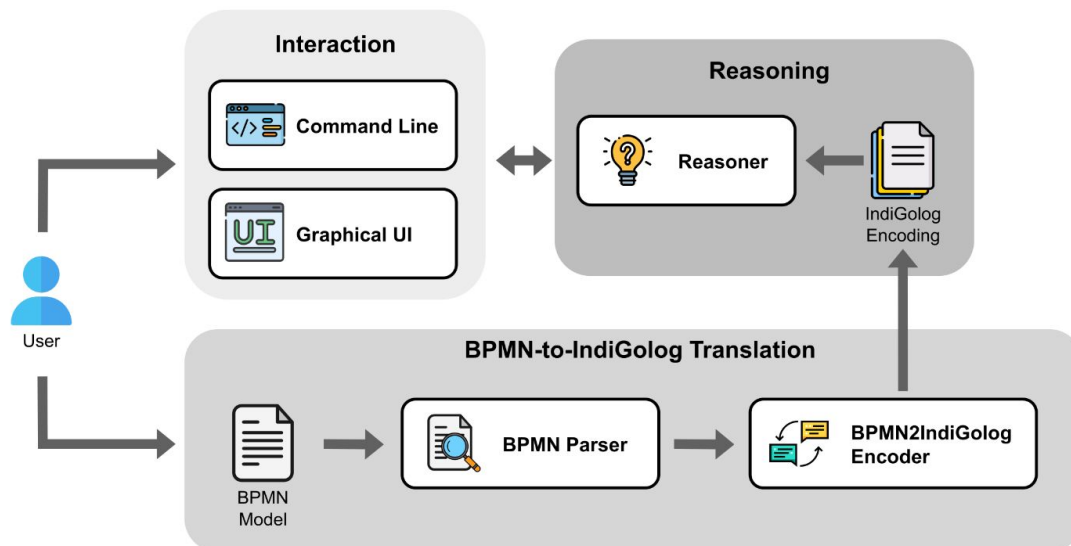
GitHub



Translation: BPMN parser + encoder generate the situation calculus theory and ConGolog program

Reasoning: IndiGolog interpreter runs projection, legality, conformance, verification

Interaction: a CLI for integration and a Gradio-based GUI.



Evaluating Translation and Reasoning

- 99 BPMN models across two datasets: 89 control-flow-heavy (Textual), 10 data/collaboration-rich (Exams).
- *Translation stays fast*: avg 4.5 ms (Textual) – 5.18 ms (Exams), scaling with model size.



100% (Acc.)
Projection

F1 100%

1 ms avg

Exams dataset



91.1% (Acc.)
Legality

F1 91.1%

26 ms avg

Textual dataset



88.3% (Acc.)
Conformance

F1 87.0%

50 ms avg

Textual dataset



94.1% (Acc.)
Verification

F1 92.3%

1 s avg

3/20 timeouts

Exams dataset

- R.M. Dijkman et al. Semantics and analysis of business process models in BPMN, Inf. Softw. Technol. 50 (12) (2008).

CONCLUSION

Contributions



- Toward a **reasoning layer** for **AI-augmented BPMNs**.
- BPMN→ConGolog formalization (+ new **guarded execution** construct).
- Support for four reasoning tasks (projection, legality, conformance, verification).
- Open-source suite for translation and reasoning.



Limitations & Next Steps



- No inclusive OR-gateway, lanes, timers, or compensation yet.
- Assumes “well-structured” BPMN models.
- Property verification can hit state-space explosion under heavy concurrency.



Business Process Management 2026

Toronto, Canada

September 29, 2026

THANK YOU

Presented by Angelo Casciani

Mail: casciani@diag.uniroma1.it

Journal

Paper



*Information Systems,
v. 140, 2026, 102718*

GitHub



About Me



SAPIENZA
UNIVERSITÀ DI ROMA



Finanziato
dall'Unione europea
NextGenerationEU



Ministero
dell'Università
e della Ricerca



Italiadomani
PIANO NAZIONALE
DI RIPRESA E RESILIENZA

Related Work

Reference	Formalism	Data	Glob. Ex.	Collab.	Analysis focus	Expressive power & limitations
Dijkman et al. [3]	Petri nets	✗	●	✗	Structural (Control-flow soundness)	(+) Deadlock and reachability check. (-) No data; limited exception handling.
Eid-Sabbagh et al. [14]	Open Nets/CTL	✗	✗	✓	Architecture analysis (interaction soundness)	(+) Interdependencies via triggers/messages. (-) No data values; limited control logic.
Meyer et al. [15]	Relational/SQL	✓	✗	✗	Enactment & data dependencies	(+) Complex data dependencies. (-) No collaboration.
Kossak et al. [16]	ASM	✓	✓	✓	Simulation	(+) Extensive BPMN coverage. (-) Verification requires transformation; higher complexity.
Kheldoun et al. [17]	Recursive ECATNets	✓	●	✓	Verification (LTL/Maude)	(+) Handles data & cancellation. (-) Complex mapping rules.
Houhou et al. [12]	FOL/TLA+	✗	✗	✓	Verification (safety, soundness)	(+) OR-gateways and communication. (-) Data abstracted.
Corradini et al. [4]	Op. Semantics (LTS)/S3	✗	✓	✓	Structural classification (safety, soundness)	(+) Arbitrary topologies, terminate events, and sub-processes. (-) Data abstracted. State explosion.
Corradini et al. [13]	Op. Semantics/Maude	✗	✗	✓	Verification (LTL & Statistical)	(+) OR-gateways & large state spaces. (-) No data support.
Corradini et al. [18]	Op. Semantics (LTS)	✓	✗	✓	Animation & Debugging	(+) Multi-instance data & correlations. (-) Simulation only.
Our approach	SitCalc/CONGolog	✓	✓	✓	Semantic reasoning	(+) Native data fluents and global exceptions. (-) No OR-gateway; structured models assumed.

Evaluation Setup: Datasets Composition

Average metrics of the BPMN process models contained in the datasets considered for evaluation.

Dataset	Pools	Tasks	Sub-processes	XOR	AND	Events	Data objects	Total elements
<i>Textual</i>	1.0	14.1	0.0	7.4	2.7	2.8	0.5	27.5
<i>Exams</i>	2.2	12.0	1.1	4.7	1.7	19.0	8.8	47.3

Evaluation Results

Average translation metrics for the BPMN datasets.

Dataset	Avg. time (ms)	Avg. actions	Avg. fluents	Avg. size (LOC)
<i>Textual</i>	4.50	34.3	8.4	261.7
<i>Exams</i>	5.18	40.3	9.6	318.1

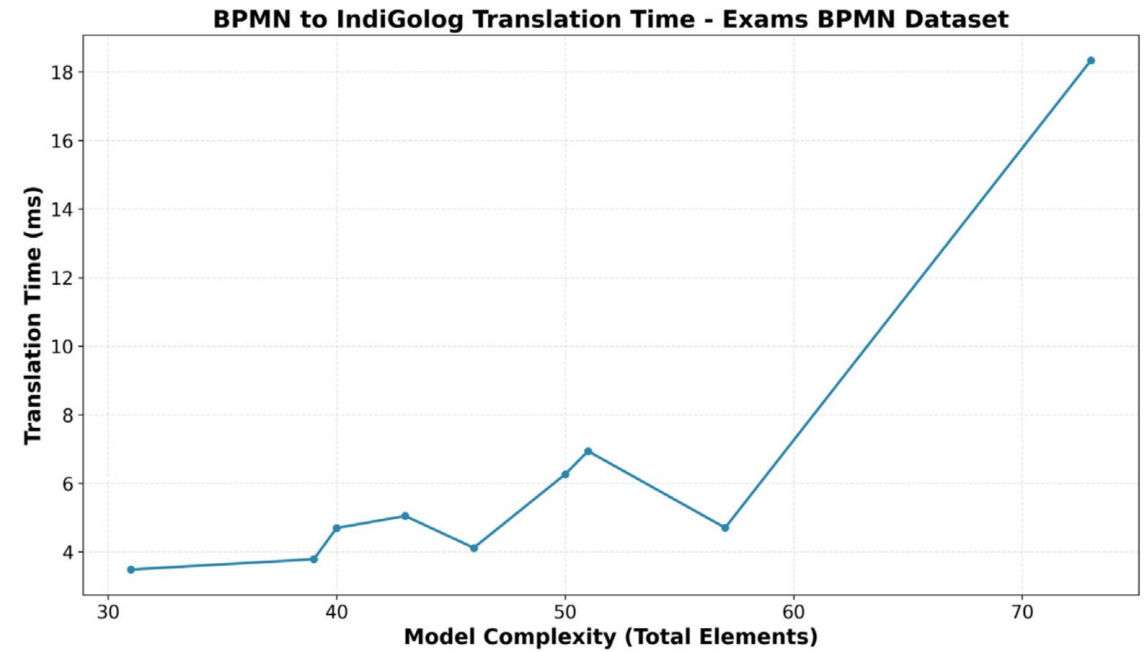
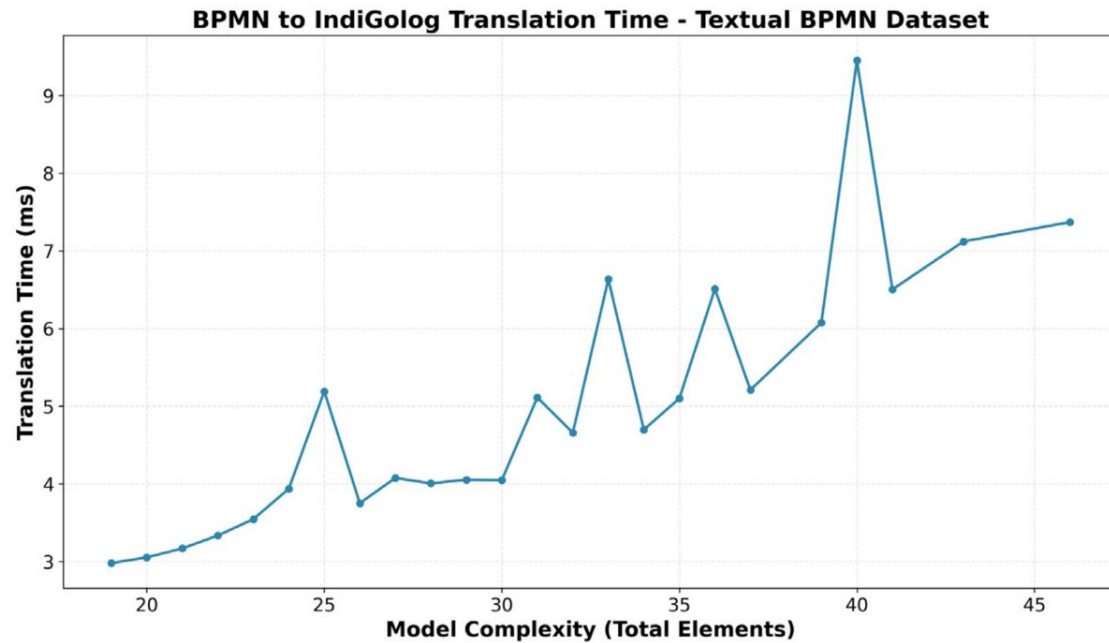
Average performance results for the control-flow evaluation on the Textual BPMN dataset. The reasoning time is reported in *seconds*.

Task	Evaluated samples	Timeout	Accuracy	Precision	Recall	F1-score	Avg. time	Avg. inferences
<i>Legality</i>	384/384	0/384	91.1%	92.0%	90.1%	91.1%	0.026	114,954
<i>Conformance</i>	384/384	0/384	88.3%	97.4%	78.6%	87.0%	0.050	662,876

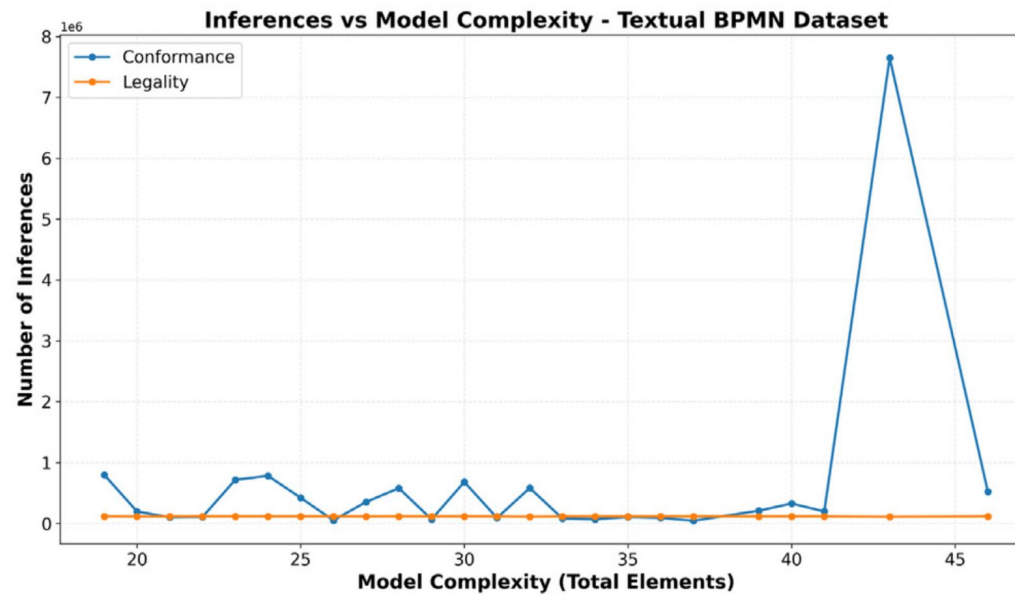
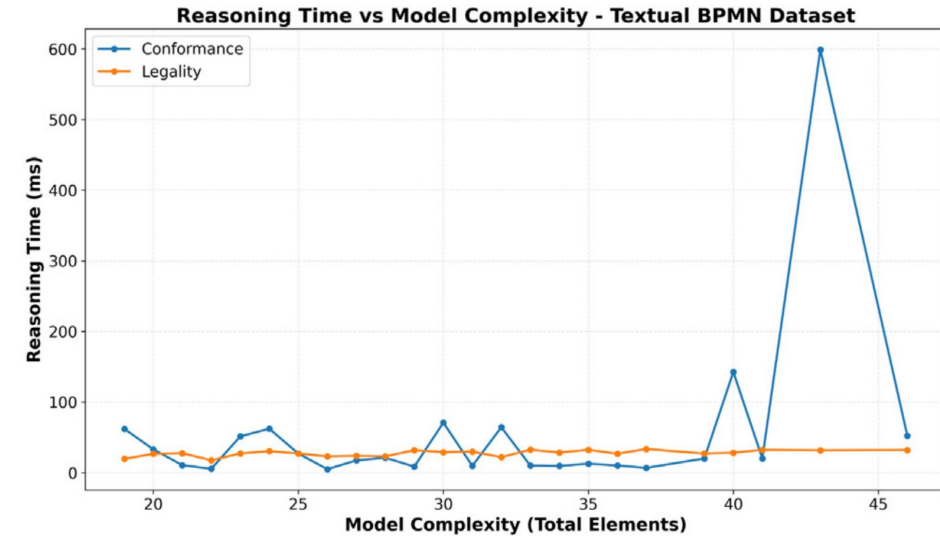
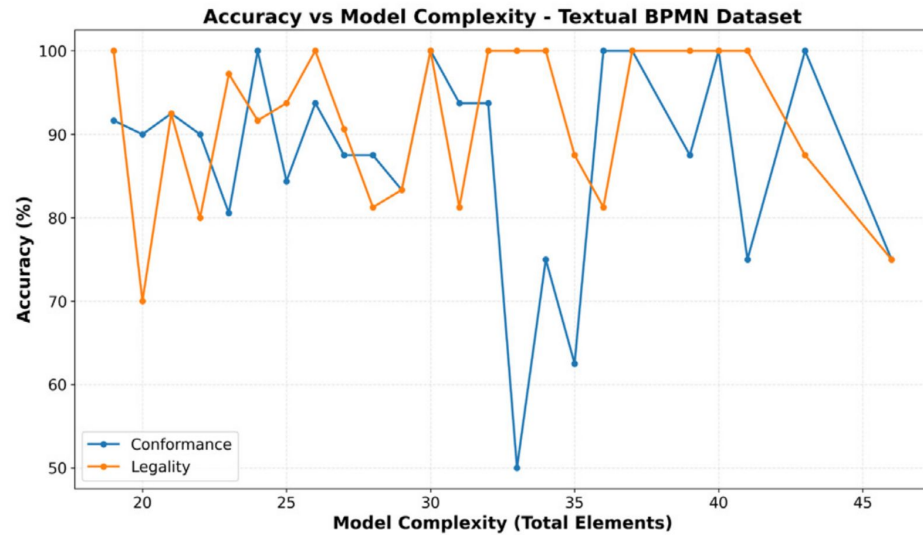
Average performance results for the data-property evaluation on the Exams BPMN dataset. The reasoning time is reported in *seconds*.

Task	Completed samples	Timeout	Accuracy	Precision	Recall	F1-score	Avg. time	Avg. inferences
<i>Projection</i>	20/20	0/20	100.0%	100.0%	100.0%	100.0%	0.001	2488
<i>Verification</i>	17/20	3/20	94.1%	100.0%	85.7%	92.3%	1.002	16,247,482

Evaluating Translation



Evaluating Reasoning - 1



Evaluating Reasoning - 2

